UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
TRUONG DAI HOC KHOA HOC VA CONG NGHE HA NOI

Bachelor Internship Report

Design and Evaluation of an AI Model Logging and Monitoring System Using the Loki Stack

A PCB AOI Case Study

Student: Do Hung Anh
Student ID: 23BI14015
Program: Information and Communication Technology Program
External Supervisor: Eng. Pham Xuan Long
Internal Supervisor: Dr. Kieu Quoc Viet
Host Organization: FUYU Precision Component VN (Foxconn Industrial Internet)

Hanoi, 2026

Design and Evaluation of an AI Model Logging and Monitoring System Using the Loki Stack

A PCB AOI Case Study

Bachelor Internship Report

Abstract

This report presents the design and evaluation of a logging and monitoring system for AI model inference services using the Grafana Loki stack. The study is grounded in an industrial Automatic Optical Inspection (AOI) workflow for printed circuit board (PCB) review, where operational visibility is as important as prediction output. A repository-aligned case study is used in which a FastAPI-based AOI backend produces structured inference events, a React workstation supports human review, and a Loki–Promtail–Grafana stack captures and visualizes system behavior. The report is organized into six chapters covering motivation, related work, system design, implementation, experiments, and evaluation. The proposed system emphasizes structured JSON logs, defect-level traceability, latency observation, historical queryability, and anomaly detection for normal, failure, and degraded inference conditions. The resulting document provides a practical foundation for demonstrating how an AI inference service can be monitored as an operational system rather than treated as an isolated model artifact.

Keywords: AI monitoring, AOI, PCB inspection, inference logging, Loki, Grafana, Promtail, FastAPI, observability, industrial AI

1 Introduction

Automatic Optical Inspection (AOI) is an industrial quality-control process in which cameras and software are used to inspect manufactured products automatically. In electronics manufacturing, AOI is widely used to inspect Printed Circuit Boards (PCBs), which are the boards that mechanically support and electrically connect electronic components such as resistors, capacitors, integrated circuits, and connectors. In a PCB production line, AOI systems help detect visible problems before defective boards continue to later assembly or shipping stages.

Typical PCB inspection examples include missing components, component misalignment, solder bridges, insufficient solder, incorrect part placement, and unusual board-level failure patterns. In modern production environments, these checks are increasingly supported by AI-based inference services that classify or localize suspicious regions on board images. Once AI is introduced into the inspection workflow, however, the challenge is no longer limited to prediction quality. It becomes a system-engineering problem in which operators and engineers must know whether the inference service is healthy, responsive, consistent, and traceable during operation.

Artificial intelligence models are often evaluated through offline metrics such as accuracy, precision, recall, or mAP. While those metrics are necessary, they are not sufficient once the model becomes part of a running system. In a real inspection line, operators

and engineers need to know not only whether a model can produce predictions, but also whether the service remains responsive, whether failures are traceable, and whether abnormal behavior can be detected quickly enough to prevent operational disruption.

This requirement is especially important in AI-assisted AOI systems for PCB inspection. AOI services interact with image acquisition, run management, defect review, and operator decisions. A defect result without metadata such as timestamp, board identity, latency, confidence, or review outcome provides only partial value. In practice, inference must therefore be observable as an operational process rather than treated only as a prediction output.

The central argument of this report is that an AI model logging and monitoring system should be designed as a first-class capability. Instead of treating logs as an afterthought, the system should generate structured events that record what happened, when it happened, how long it took, and what downstream consequence it produced. This makes the AI pipeline easier to audit, debug, benchmark, and improve.

The study uses a repository-aligned AOI workstation as the case environment. The software stack includes a FastAPI backend, a React review interface, local storage, JSONL event logging, and a Docker-based observability stack with Loki, Promtail, and Grafana. The monitoring goal is to support real-time observation, historical log search, performance-trend analysis, and detection of three broad operating states: normal behavior, explicit failure behavior, and degraded high-latency behavior.

1.1 Problem Context: What AOI Monitoring Must Support

For this report, the practical object under study is not only the PCB detector itself, but the operational layer around it. In an AOI environment, a useful monitoring system should support at least four needs:

- visibility into how many inference events are being produced;
- traceability of which PCB or component triggered a failure;
- observability of latency, confidence, and abnormal prediction trends; and
- support for operator review and post-incident investigation.

Without these capabilities, an AOI system may still generate predictions, but it remains difficult to trust, debug, or operate reliably in an industrial setting.

1.2 Research Framing: What, Why, How, and Results

This report is structured around four guiding questions.

What is being built? An AI inference logging and monitoring system for a PCB AOI workflow, using structured event logging and the Loki–Promtail–Grafana stack.

Why is it needed? Because AI-assisted inspection systems require operational visibility in addition to model accuracy. Engineers and operators need real-time monitoring, historical log search, and anomaly detection to maintain reliability.

How is it implemented? Through a repository-aligned case study that combines a FastAPI backend, a React review workstation, JSONL event logs, Promtail for log shipping, Loki for aggregation and querying, and Grafana for dashboards.

What results are expected? The system should make it possible to distinguish healthy behavior from failure spikes and latency degradation, while also preserving traceable evidence for evaluation and debugging.

1.3 Objectives

The objectives of this report are as follows:

- explain why inference monitoring matters in operational AI systems;
- review observability practices relevant to ML services;
- design a monitoring architecture suitable for a local AOI platform;
- describe a concrete implementation grounded in the PCB AOI project;
- propose experiments that simulate multiple runtime conditions; and
- evaluate whether the system supports practical anomaly detection.

1.4 Why This Work Is a Suitable Bachelor Internship Topic

This topic is appropriate for a bachelor internship because it combines several important skills in one applied engineering problem: backend integration, structured data design, industrial software workflow understanding, dashboard construction, and system evaluation. The work is also concrete and bounded. It does not attempt to solve all of AOI or all of machine learning. Instead, it focuses on a realistic and defensible contribution: making an AI-based AOI service observable and evaluable in practice.

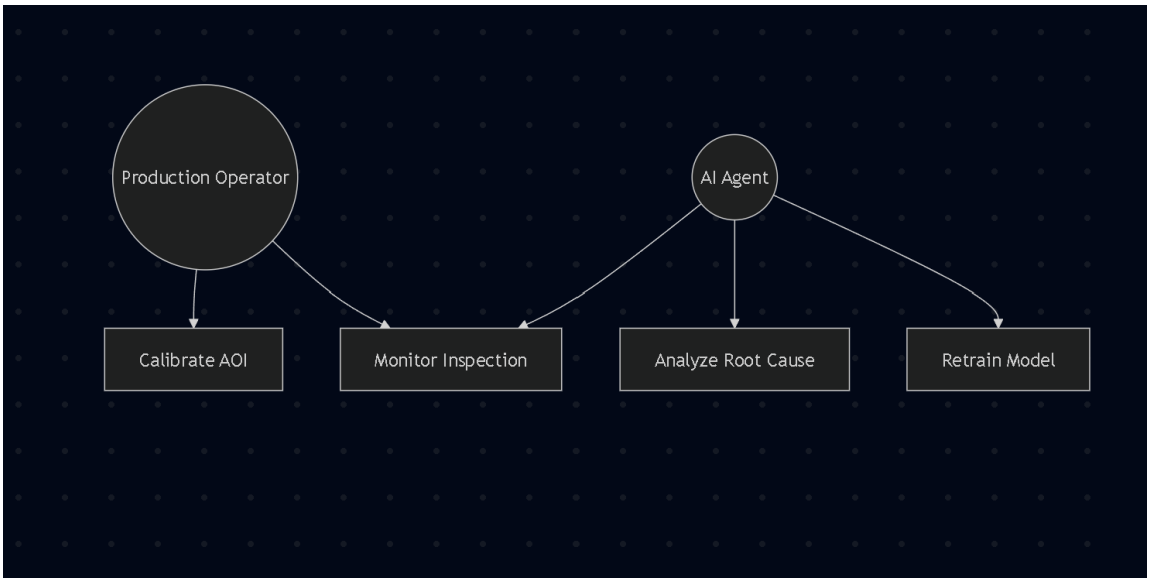

Figure 1: Conceptual AOI monitoring loop linking operator decisions, inspection monitoring, root-cause analysis, and model improvement.

2 Related Work

2.1 Observability in Software Systems

Observability is a core idea in distributed systems and production software. Traditional monitoring emphasizes aggregate counters and infrastructure health, while observability aims to make internal system behavior understandable through logs, metrics, and traces. In AI systems, this distinction is important because model-serving failures may appear even when CPU, memory, or network indicators seem normal.

2.2 Monitoring Stacks Based on Logs

The Loki stack is a pragmatic logging solution for lightweight deployments. Loki aggregates logs, Promtail ships them from services, and Grafana provides queries, dashboards, and alert-friendly visualization. Compared with heavyweight full observability platforms, this stack is attractive for report-scale or prototype environments because it is easy to deploy locally and integrates well with structured JSON logs.

2.3 ML and MLOps Monitoring

Monitoring in machine learning systems usually covers several categories: system health, inference latency, prediction distribution, data drift, model versioning, and human feedback loops. Large MLOps platforms often include these features, but smaller projects frequently need a narrower and more direct approach. For a local AOI workstation, the most relevant signals are event volume, board throughput, failure frequency, confidence patterns, and per-event latency.

This report is also informed by a broader software-engineering literature on ML systems. Sculley et al. argue that production ML systems accumulate system-level technical debt through entanglement, hidden feedback loops, undeclared consumers, and brittle interfaces rather than only through poor model performance. Breck et al. propose the ML Test Score as a production-readiness rubric, emphasizing data validation, infrastructure testing, and monitoring discipline around deployed models. Amershi et al. show, through an industrial case study at Microsoft, that developing ML-enabled products requires a wider process change than conventional software engineering, especially around data, versioning, and integration. Shankar et al. extend this discussion by arguing that production ML pipelines require end-to-end observability because silent failures often occur outside the model itself. These works strengthen the motivation for a thesis that treats AOI inference as an operational system to be monitored, not only a predictor to be benchmarked.

2.4 Monitoring in Industrial Inspection

Industrial inspection systems differ from generic web services because they are tightly connected to physical workflows. Missing logs can make defect review non-reproducible, while unstable latency can slow operator decisions. This means monitoring must support both engineering needs and operational review. The literature and industry practice converge on the same lesson: trustworthy inspection requires traceability.

2.5 Comparison With Large-Scale Monitoring Platforms

Large industrial AI systems often operate at a scale far beyond a bachelor internship project, and it is useful to state this difference clearly. Netflix reports that its Mantis platform has been in production since 2014 and processes trillions of events and peta-bytes of data every day. The same public documentation describes use cases that analyze data from thousands of servers, millions of interactions across dozens of microservices, and hundreds of Cassandra and ElasticSearch clusters. In separate Netflix platform material, the company also reports tens of thousands of running instances across its deployment environment.

This report does not attempt to replicate that scale. Instead, it adopts a similar engineering principle at a much smaller scope: preserve structured raw events, keep the monitoring pipeline queryable in near real time, and make it possible to ask new operational questions without changing the application each time a new anomaly is suspected.

It is also useful to compare the present design with specialized inference servers such as NVIDIA Triton. Triton exposes a rich metrics surface including request duration, queue duration, inference count, and execution count. Those capabilities are important in high-throughput GPU-serving environments. By contrast, the AOI workstation in this report is optimized for local deployability, defect traceability, and end-to-end observability of an AOI review workflow rather than raw serving throughput. The tradeoff is deliberate: FastAPI plus structured JSONL logging is easier to audit, easier to modify, and better matched to a repository-scale case study than a full production-grade inference platform.

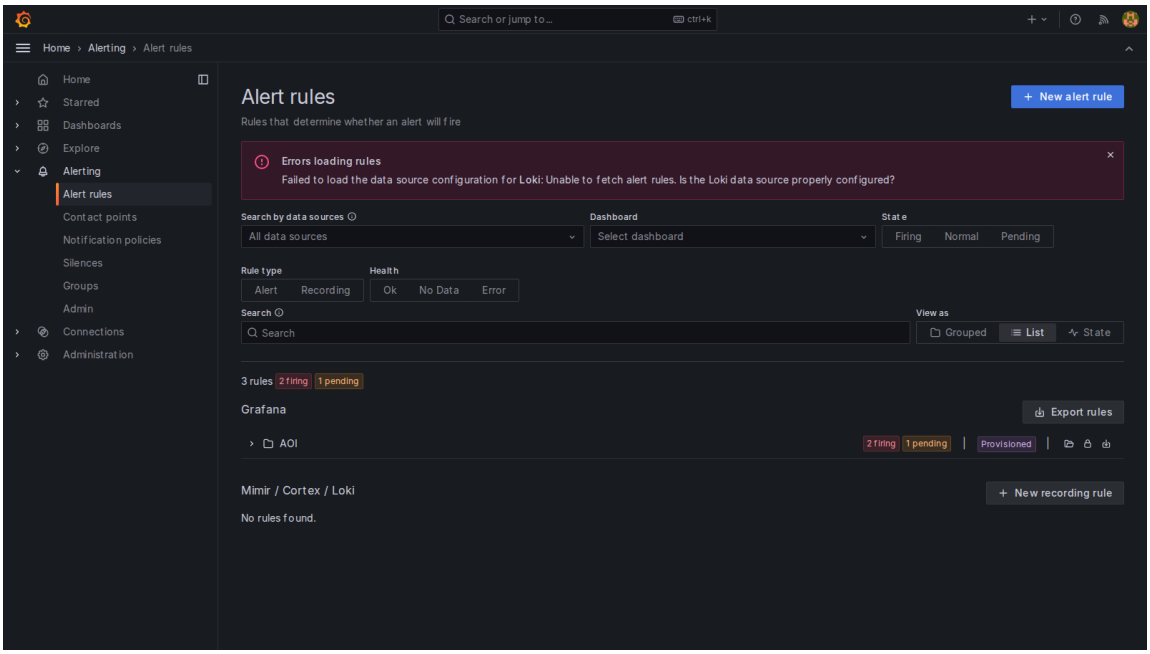

Figure 2: Provisioned alert rules demonstrate the transition from passive observability to active monitoring.

3 System Design

3.1 Design Principles

The proposed monitoring architecture follows five principles:

1. structured logs instead of free-form strings;
2. event schemas aligned with AOI domain entities;
3. separation between inference, persistence, and visualization;
4. local deployability for development and report demonstration; and
5. support for anomaly detection through simple, inspectable queries.

3.2 Architecture Overview

The architecture begins with the AOI application, which emits inference events as structured JSON entries. Each event may include a PCB identifier, component identifier, inspection result, defect type, confidence score, latency, and normalized overlay geometry. These logs are written to a file that Promtail scrapes and forwards to Loki. Grafana then queries Loki to present operational dashboards such as total events, recent failures, boards processed, and latency trends.

In repository terms, the path is concrete rather than conceptual. A FastAPI API accepts `POST /events` requests, writes event rows to JSONL, and stores run metadata in SQLite. Promtail scrapes `/var/log/aoi/*.jsonl`, extracts selected fields from each JSON object, and promotes a subset of those fields to labels. Loki stores the resulting log streams, while Grafana provides the dashboard and alerting surface. This design keeps the event source, log transport, query model, and visual layer clearly separated.

3.3 Event Model

The event schema is the core monitoring contract. An effective schema should capture both model output and operational metadata. In the AOI case, the most important fields are:

- `timestamp` for temporal ordering;
- `pcb_id` for board-level grouping;
- `component_id` for defect localization;
- `inspection_result` for pass/fail state;
- `defect_type` for defect categorization;
- `confidence_score` for prediction strength;
- `inference_latency_ms` for performance monitoring; and
- overlay coordinates for frontend review traceability.

This field selection is intentionally narrower than a full cloud MLOps schema. The goal is to preserve the signals most relevant to AOI operation while keeping queries simple. In the Promtail pipeline, `pcb_id`, `component_id`, `inspection_result`, and `defect_type` are promoted to labels because they are frequently used for filtering and grouping. By contrast, `confidence_score` and `inference_latency_ms` remain JSON values that are parsed on demand through LogQL `| json | unwrap ...` expressions. This reduces label cardinality while preserving numeric analysis.

3.4 Dashboard Logic

The dashboard is designed around operational questions:

- How many events arrived in the last five minutes?
- How many recent detections were failures?
- How many boards were processed in a recent time window?
- Is inference latency stable or increasing?

These questions map directly to Loki queries and make the dashboard useful for both live monitoring and retrospective investigation.

3.5 Anomaly Types

The design targets three anomaly classes:

- absence or drop in expected event traffic;
- elevated failure concentration; and
- latency spikes indicating degraded inference service behavior.

This is intentionally simple. The goal is not to claim full MLOps maturity, but to build a robust foundation that supports future alerting and model analytics.

3.6 Why This Design Pattern Was Chosen

The monitoring architecture follows a log-centric MLOps pattern for four practical reasons.

1. **Low integration cost.** A FastAPI service can emit structured JSON without introducing a specialized model server or a metric exporter at the first stage of the project.
2. **High diagnostic value.** A single event record can carry both operational fields such as latency and AOI-specific fields such as `pcb_id` and `defect_type`. This is harder to capture in aggregate metrics alone.
3. **Observability as code.** Promtail parsing rules, Grafana dashboards, and alert thresholds are provisioned from version-controlled files, which makes the monitoring behavior reproducible.

4. **Fit to scope.** The thesis question is whether anomalies in an AOI inference service become visible and actionable. It is not whether the repository can outperform industrial serving platforms on throughput.

The main alternative would be a production inference stack centered on systems such as Triton, KServe, or a larger stream-processing platform. Those architectures are stronger when dynamic batching, GPU multiplexing, autoscaling, or multi-tenant serving are the central problem. In this project, however, the critical problem is explainable monitoring around a bounded AOI workflow. The chosen pattern therefore prioritizes interpretability over maximum serving complexity.

3.7 MLOps and Observability-as-Code Patterns

Although the implementation is lightweight, it still follows several recognizably MLOps-oriented patterns:

- **Schema as contract:** the event structure standardizes what an inference result must contain before it enters the monitoring stack.
- **Logs as canonical evidence:** every experiment ultimately becomes a sequence of JSONL records that can be replayed, queried, and audited.
- **Infrastructure as code:** alert rules and scrape pipelines are defined in YAML rather than configured manually in a UI.
- **Experiment harness separation:** scenario generation, sequential execution, and report extraction are implemented as separate modules rather than one ad hoc script.

4 Case Study Implementation

4.1 Repository-Aligned AOI Platform

The case study implementation is based on the current AOI repository. The backend is implemented in Python with FastAPI and exposes routes for health, runs, and event ingestion. The frontend is a React workstation for PCB review, including run navigation, image viewing, overlay display, and operator review controls. SQLite is used for local persistence, and JSONL is used for event log output.

4.2 Implemented Monitoring Components

The implementation contains the following concrete monitoring-relevant modules:

- a schema layer defining `InferenceEvent` and inspection states;
- a log manager that appends JSON records to persistent log files;
- a mock inference generator for controlled event simulation;
- a Docker Compose stack containing Loki, Promtail, and Grafana; and

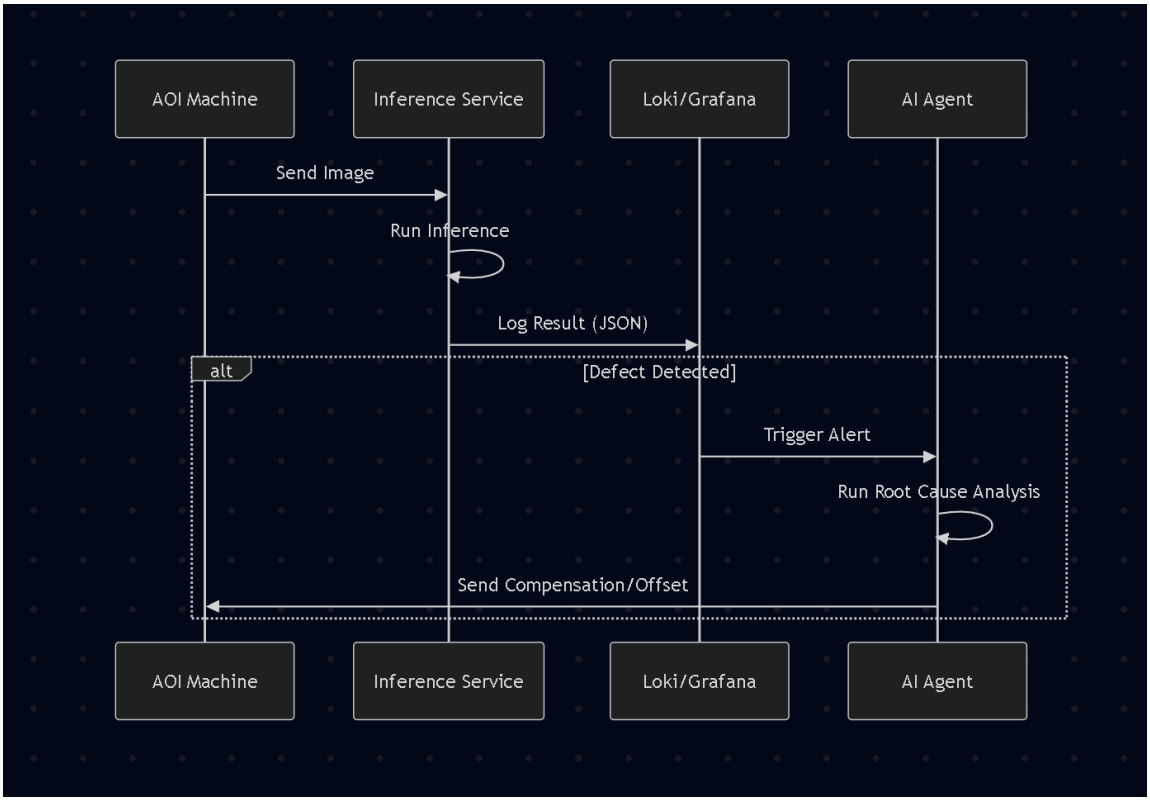

Figure 3: Repository-aligned architecture for structured event logging and anomaly monitoring.

- a provisioned Grafana dashboard for AOI event inspection.

Two implementation details are especially important. First, the Promtail configuration parses JSON and promotes domain identifiers to labels. This is what makes queries such as `{job="aoi-inference", pcb_id= "PCB-DEAD-.*"}` possible without full-text search. Second, the alert rules are provisioned as code in Grafana, with three explicit threshold policies: fail rate > 0.60, average latency > 500 ms, and average confidence < 0.60. Each rule follows the same pattern: a Loki range query, a reduce stage, and a threshold stage.

4.3 Event Flow in the AOI Service

The service receives or generates inference-like outputs and converts them into structured events. A typical event includes board identity, component identity, inspection result, defect type, confidence score, latency, and optional defect overlay coordinates. The backend can then persist the event and expose related data to the review workstation. This creates a direct path from AI output to human inspection and to observability tooling.

4.4 Why the AOI Project is a Good Monitoring Case

The AOI project is appropriate for monitoring research because it combines image processing, event logging, persistence, operator review, and dashboarding within a single system. It also includes synthetic event generation, which is useful for testing failure

scenarios without requiring a fully deployed production model. This makes the platform suitable for controlled experimentation and report demonstration.

4.5 Code Pattern: Event Construction and Safe Emission

The experiment harness avoids raw dictionary duplication by centralizing event construction in `experiments/scenarios/base.py`. The helper function `make_event(...)` clamps confidence to the API's accepted range, prevents negative latency values, and only emits overlay geometry when the inspection result is `FAIL`. This is a useful pattern because it keeps scenario scripts focused on behavior rather than payload plumbing.

Listing 1: Core event-construction pattern in `experiments/scenarios/base.py`.

```
def make_event(...):
    confidence_score = max(0.0, min(1.0, confidence_score))
    inference_latency_ms = max(0, inference_latency_ms)
    event = {
        "pcb_id": pcb_id,
        "component_id": component_id,
        "inspection_result": inspection_result,
        "defect_type": defect_type,
        "confidence_score": confidence_score,
        "inference_latency_ms": inference_latency_ms,
        "overlay_shape": "rect" if inspection_result == "FAIL" else None,
    }
    return {k: v for k, v in event.items() if v is not None}
```

The batch sender in the same module provides a second useful pattern: scenario scripts prepare batches only, while HTTP posting, timing, and error collection are handled centrally. This design reduces code duplication and guarantees that all nine scenarios are measured in the same way.

Table 1: Case Study Components and Monitoring Roles

Layer	**Repository Component**	**Monitoring Role**
API backend	FastAPI service	Accepts events, exposes health, supports run review
Schema	`InferenceEvent`	Defines structured monitoring payload
Logging	JSONL log manager	Persists machine-readable events
Transport	Promtail	Ships logs to aggregation layer
Storage	Loki	Indexes and queries log streams
Visualization	Grafana dashboard	Shows event volume, failures, boards, latency
Frontend	React workstation	Uses event data for review traceability

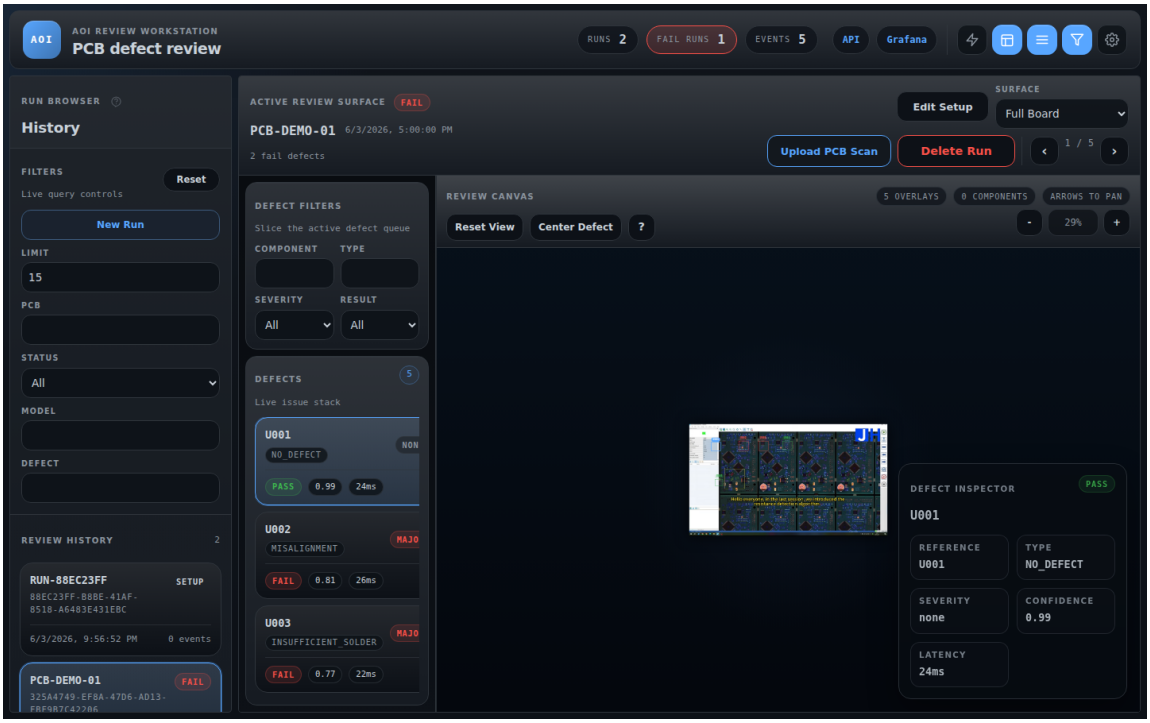

Figure 4: AOI review workstation used as the application-side consumer of the same event data.

5 Experiments

5.1 Experimental Goal

The experimental objective is to determine whether the monitoring stack can differentiate between normal and abnormal inference behavior. Because the project already supports mock event generation, experiments can be staged in a controlled way without depending on a finished production detector. This approach is appropriate for a bachelor internship because it allows repeated, explainable, and low-risk validation of monitoring behavior before full factory deployment.

5.2 Experiment Design

The anomaly-testing plan defines a sequence of controlled scenarios in which event payloads are deliberately modified to represent different operational states. Each scenario changes one important signal, such as fail rate, latency, confidence, or temporal stability, while keeping the rest of the pipeline intact. This makes it possible to observe whether Grafana panels, Loki queries, and alert thresholds respond as expected.

5.3 Experiment Harness: Build, Run, and Report

The repository separates the experiment lifecycle into three stages that a judge can inspect directly.

Build stage. Individual scenario modules such as `s02_high_defect_rate.py` and `s03_latency_spike.py` create batches of events with controlled statistical properties. For example, S02 constructs ten batches with a 9:1 fail-to-pass ratio, while S03 preserves

prediction labels but gradually raises `inference_latency_ms` from approximately 30 ms to 2000 ms. This is how the test scenes are built.

Run stage. The module `experiments/runner.py` maps scenario IDs S01–S09 to their generators and executes them sequentially. A 15-second pause is inserted between scenarios so that Grafana time-series panels show visually separated anomaly windows instead of one merged stream.

Listing 2: Scenario orchestration pattern in `experiments/runner.py`.

```
ALL_SCENARIOS = {"S01": s01_normal_baseline, ..., "S09": s09_model_degradation}
INTER_SCENARIO_PAUSE_SECONDS = 15

for idx, scenario_id in enumerate(targets):
    module = ALL_SCENARIOS[scenario_id]
    result = module.run(endpoint=endpoint)
    results.append(result)
    if idx < len(targets) - 1:
        time.sleep(INTER_SCENARIO_PAUSE_SECONDS)
```

Report stage. After execution, `experiments/report.py` queries Loki directly through its HTTP API, counts matching log entries over a lookback window, and writes a Markdown report to `docs/experiments/anomaly_results.md`. This closes the loop from test generation to runtime evidence to report artifact.

Listing 3: Loki-backed report generation pattern in `experiments/report.py`.

```
total_events = count_logs('{job="aoi-inference"}', lookback_minutes)
total_fails = count_logs('{job="aoi-inference", inspection_result="FAIL"}',
                         lookback_minutes)
fail_rate = (total_fails / total_events * 100) if total_events else 0
output_path.write_text("\n".join(lines))
```

This three-part split is a deliberate design choice. It makes the experiment suite reproducible, easier to extend, and easier to defend academically because the judge can inspect how scenarios are generated, how they are injected, and how final evidence is computed.

5.4 Scenario Set

The experiment suite contains one baseline scenario and multiple anomaly scenarios. The baseline establishes normal operating behavior, while the other scenarios test the system's ability to detect spikes, drifts, and recovery.

5.5 Baseline and Core Anomaly Cases

S01: Normal baseline. The baseline scenario sends mixed `PASS`/`FAIL` events with a roughly 70/30 ratio, high confidence values, and realistic latency in the 20–45 ms range. Its role is to establish the control condition against which all anomaly scenarios are compared. Under this scenario, the expected Grafana behavior is steady traffic, fail rate around 30%, and average latency below 50 ms.

S02: High defect rate spike. This scenario simulates a production anomaly in which more than 80% of events are failures. The monitoring question is whether the

Table 2: Anomaly Testing Scenarios

ID	Scenario	Main Monitoring Purpose
S01	Normal baseline	establish healthy reference traffic and baseline latency
S02	High defect rate spike	detect sudden fail-rate anomaly
S03	Latency spike	detect severe inference slowdown
S04	Low confidence storm	detect model uncertainty or out-of-distribution behavior
S05	Single defect-type flood	isolate systematic failure patterns by defect label
S06	Board-level failure	isolate catastrophic failures affecting specific boards
S07	System recovery	verify anomaly resolution and alert recovery
S08	Intermittent faults	detect short bursts that may be hidden in aggregates
S09	Gradual model degradation	detect long-horizon confidence decline and fail-rate growth

dashboard and alert rules can identify a sudden abnormal prediction pattern. The expected behavior is a dominant `FAIL` trend in the inspection-result panel and a triggered fail-rate alert once the proportion exceeds the configured threshold.

S03: Latency spike. This scenario preserves plausible prediction outputs while forcing `inference_latency_ms` into the 500–2500 ms range. It represents resource starvation, model loading issues, or runtime degradation. The expected observation is a strong increase in the latency panel and activation of a latency-focused alert condition.

S04: Low confidence storm. This scenario simulates model uncertainty by reducing confidence values to the 0.40–0.55 range. It is intended to represent out-of-distribution boards or other inputs for which the model becomes uncertain. The expected behavior is a clear drop in average confidence and straightforward retrieval of the affected events through LogQL filters on `confidence_score`.

5.6 Extended Cases

The remaining scenarios test more specific but still realistic operational patterns.

S05: Single defect-type flood. This scenario checks whether a single defect type, such as `SOLDER_BRIDGE`, can be isolated when it dominates all failures. Its purpose is to determine whether the log schema and Grafana panels can reveal a systematic production issue rather than random defect variation.

S06: Board-level failure. This scenario checks whether failures can be grouped and retrieved at the board level when specific PCBs fail catastrophically. It is especially useful for demonstrating the value of `pcb_id` labeling in Loki queries.

S07: System recovery. This scenario evaluates whether the system can show the full fault lifecycle: onset, peak, and return to normal operation. It tests whether the monitoring stack supports not only anomaly detection but also anomaly resolution.

S08: Intermittent faults. This scenario tests sensitivity to intermittent bursts that

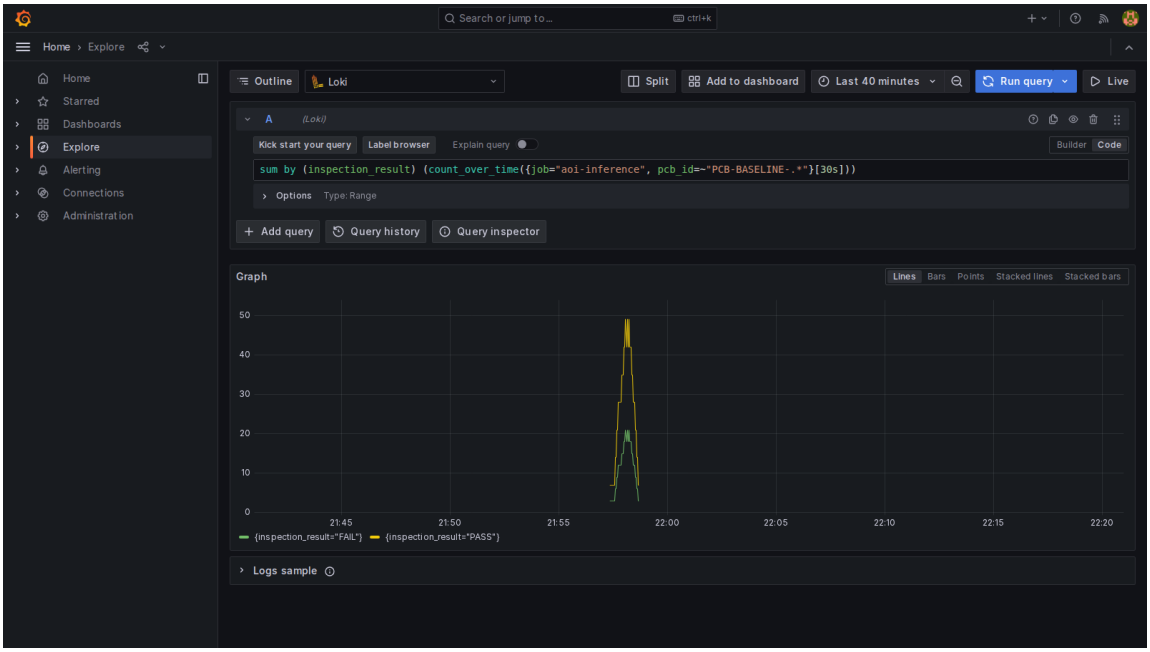

Figure 5: Scenario S01: Normal baseline dashboard view.

may be difficult to detect from coarse aggregates alone. It is intended to show the value of narrow time-window LogQL investigation.

S09: Gradual model degradation. This scenario evaluates trend detection by gradually decreasing confidence from approximately 0.95 to 0.45 while increasing fail rate over time. It represents a drift-like process rather than a sudden fault event.

S09 is the most analytically important scenario in the suite because it resembles the way many production ML failures actually emerge. Unlike S02 or S03, which cross an obvious threshold quickly, S09 degrades along two coupled dimensions: confidence falls steadily while the probability of failure rises over successive batches. This pattern is operationally significant because it may not trigger immediate human attention if operators inspect only recent single values. In the AOI context, such a trend could correspond to a slow change in incoming board characteristics, a calibration drift in upstream image capture, or a model that is becoming less suitable for the current production mix. The fact that the dashboard preserves this long-horizon shape is therefore one of the strongest arguments for using a log-centric observability pipeline rather than relying only on coarse aggregate counters.

5.7 Measurements

The proposed measurements are:

- event ingestion count over fixed windows;
- fail-event count over fixed windows;
- number of distinct boards observed;
- per-event inference latency;
- confidence trends over time;

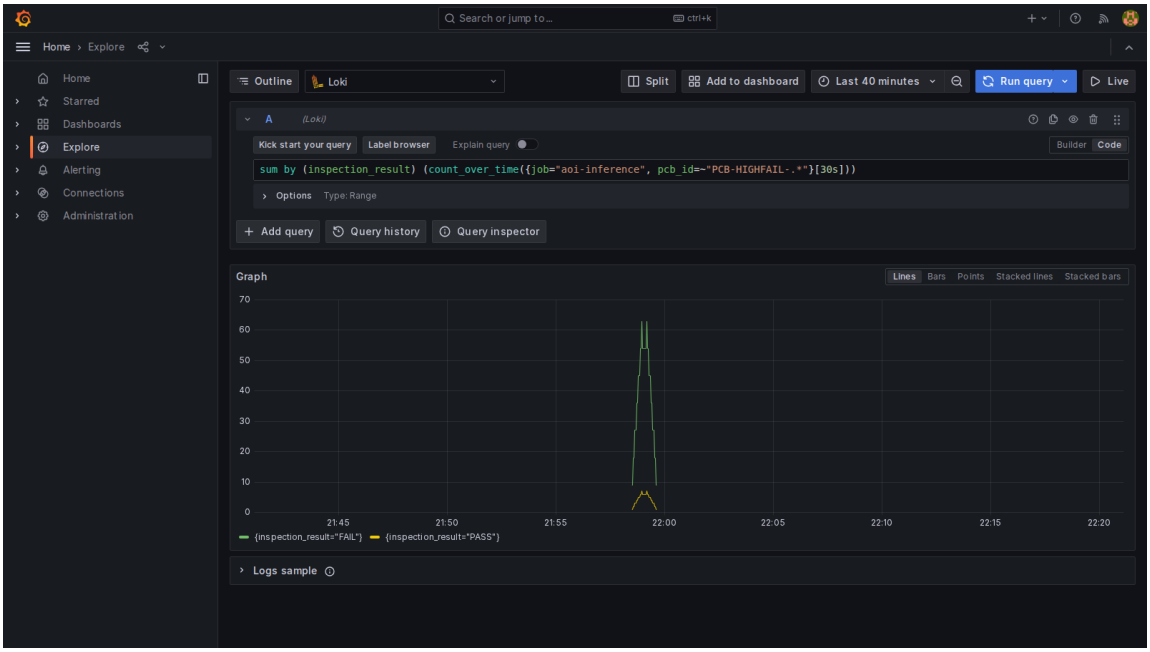

Figure 6: Scenario S02: High defect-rate spike.

- dashboard visibility of each scenario; and
- query responsiveness for recent log inspection.

Table 3: Primary Signals Used in Evaluation

Signal	**Source**	**Why It Matters**
Fail rate	event labels and LogQL counts	detects sudden shifts in prediction outcomes
Latency	`inference_latency_ms`	detects degraded runtime behavior
Confidence	`confidence_score`	indicates uncertainty or drift-like behavior
Board grouping	`pcb_id`	supports board-level isolation and traceability
Defect composition	`defect_type`	helps distinguish systematic faults from random noise

5.8 Representative LogQL Queries

To support thesis evaluation, the monitoring workflow should include a small set of representative LogQL queries that can be executed directly in Grafana Explore. These queries are not only operational tools; they also serve as evidence that the event schema is rich enough to support targeted investigation.

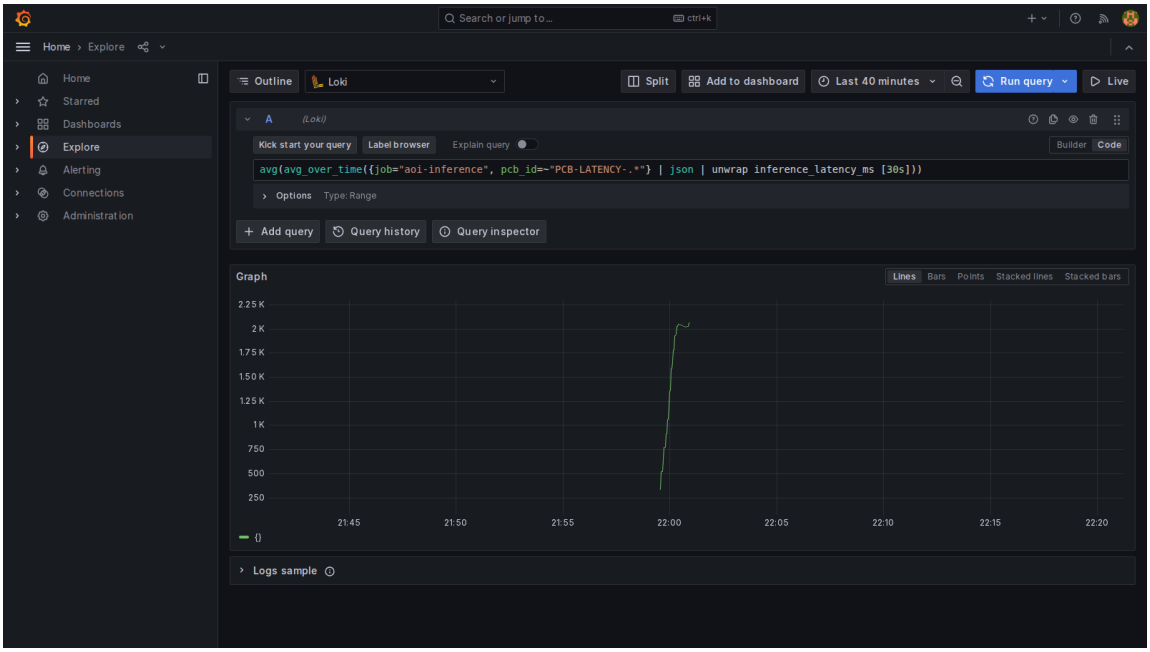

Figure 7: Scenario S03: Latency spike anomaly.

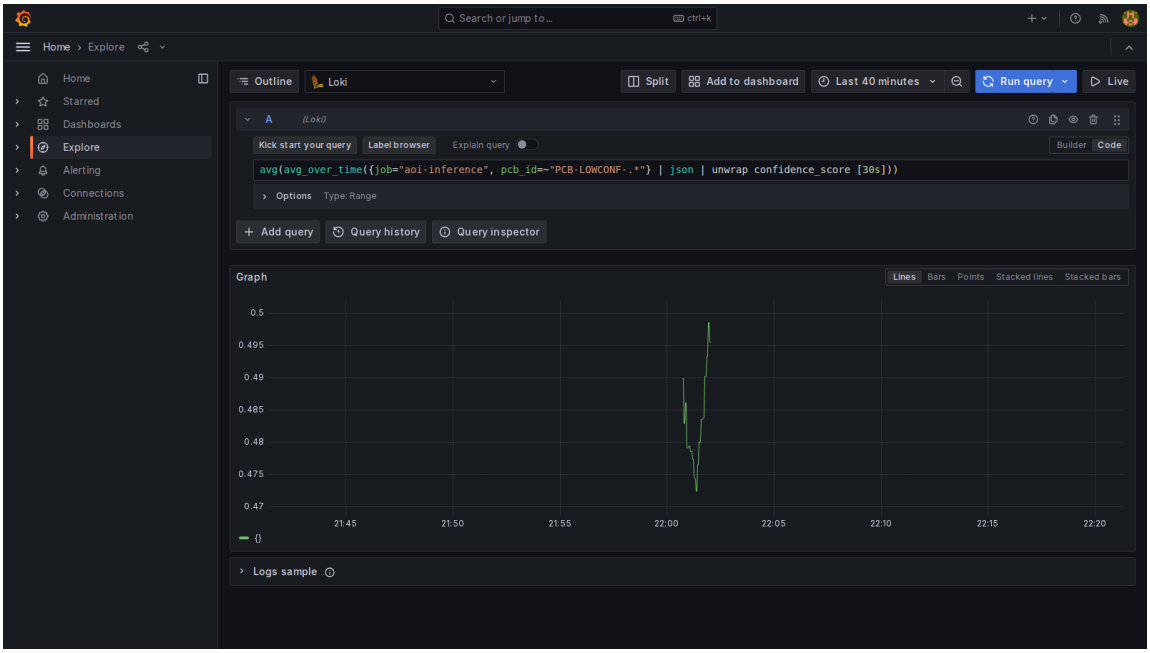

Figure 8: Scenario S04: Low-confidence storm.

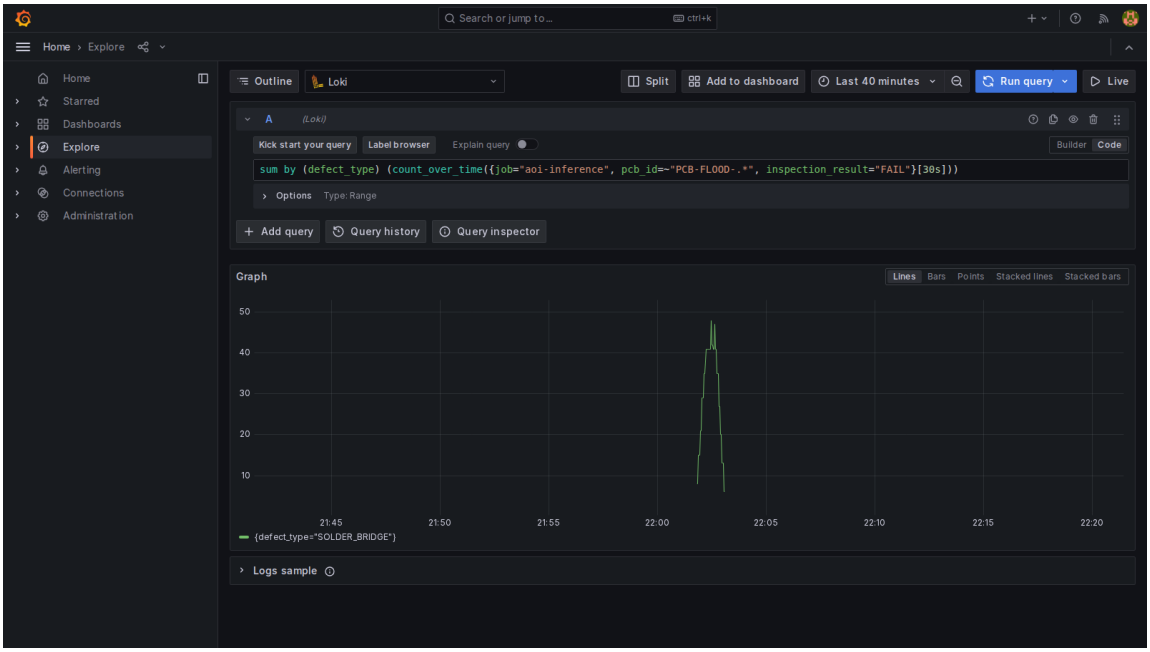

Figure 9: Scenario S05: Single defect-type flood.

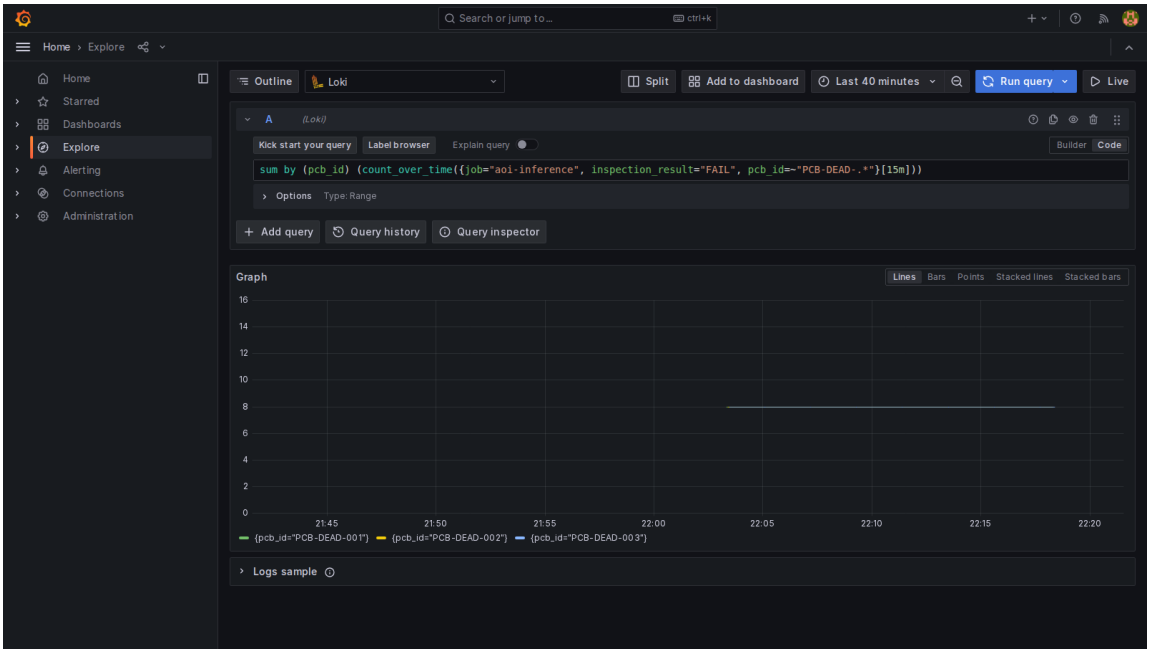

Figure 10: Scenario S06: Board-level failure isolation.

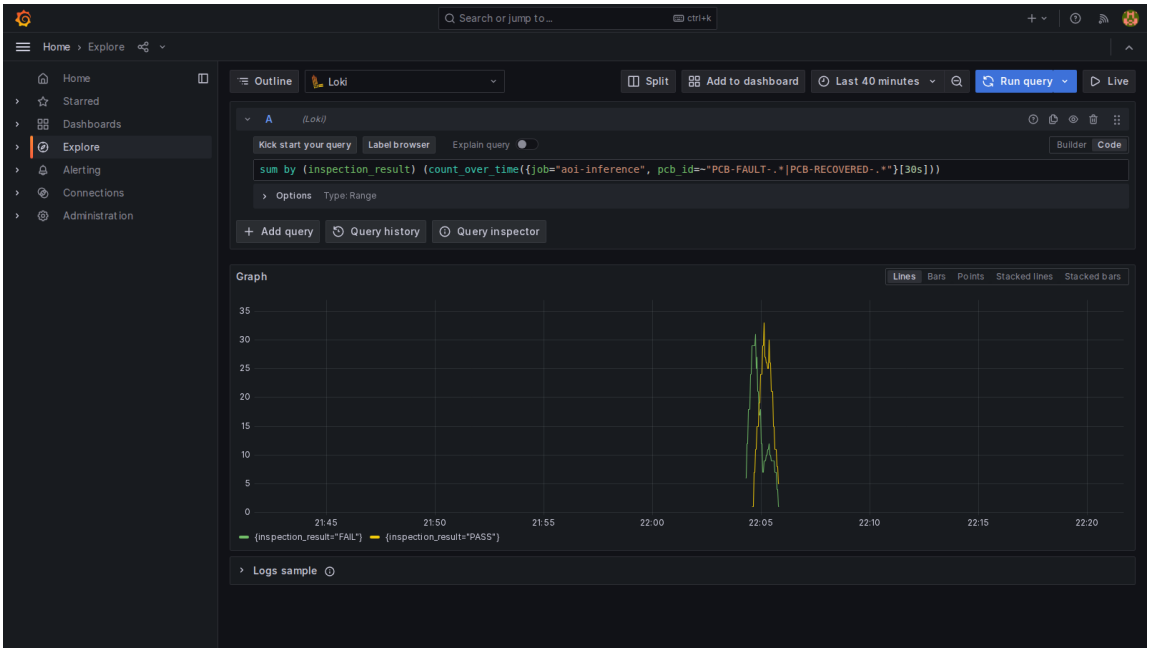

Figure 11: Scenario S07: Recovery after anomaly.

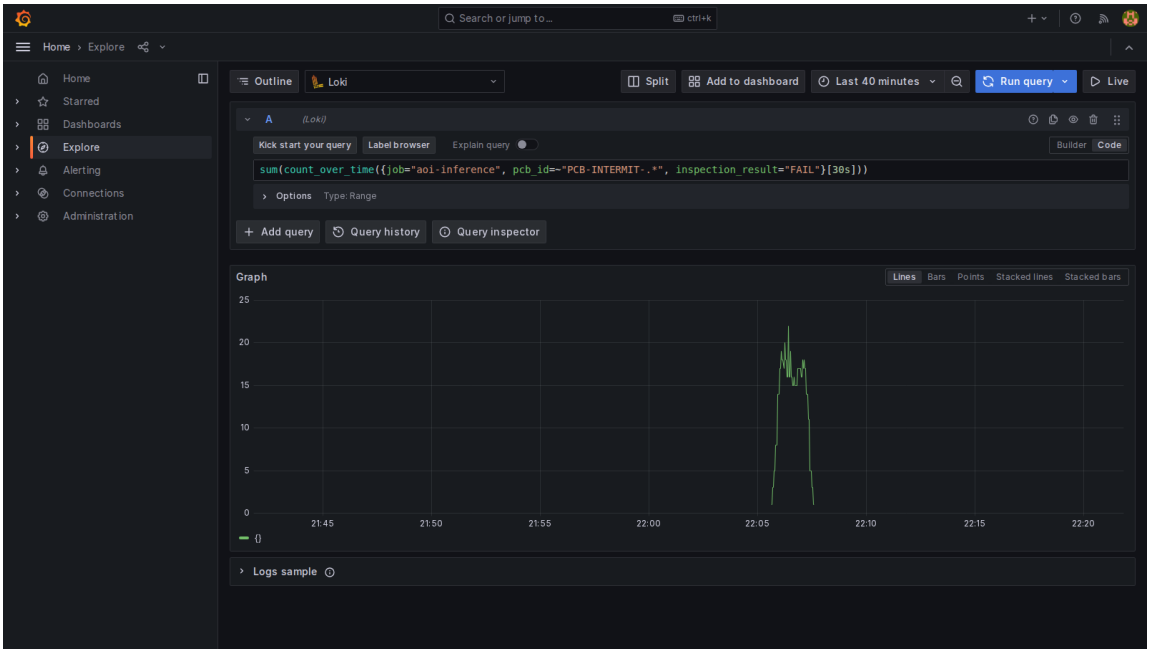

Figure 12: Scenario S08: Intermittent fault bursts.

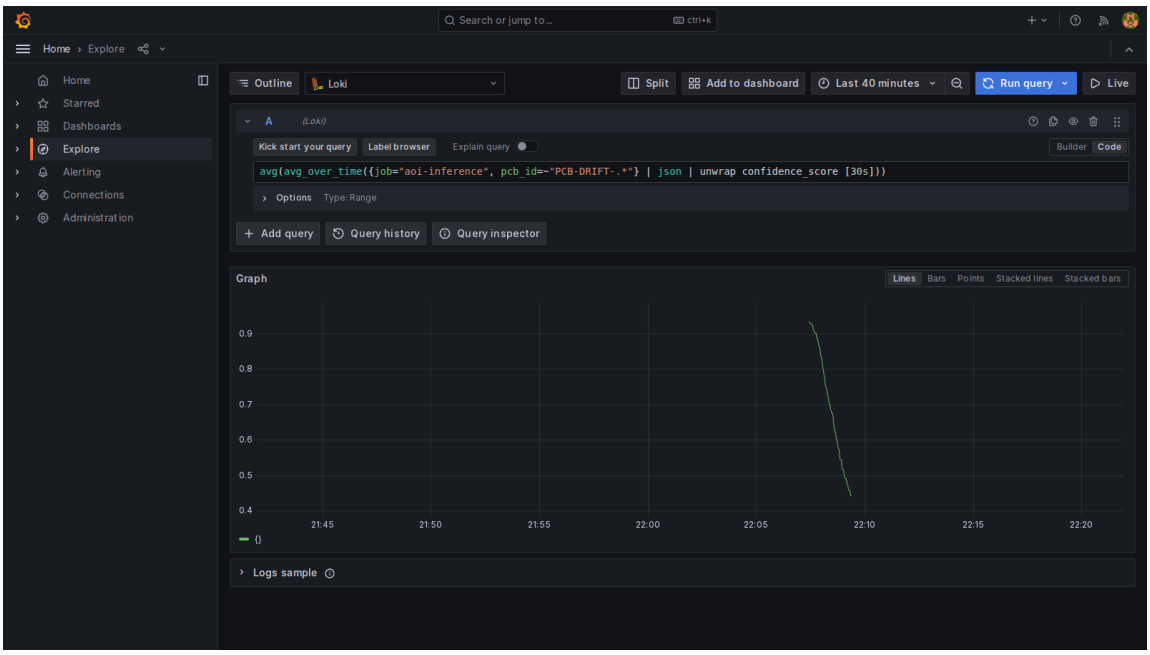

Figure 13: Scenario S09: Gradual model degradation.

Table 4: Representative LogQL Queries for Evaluation

Purpose	**Example Query**
All recent AOI events	`{job="aoi-inference"} | json`
Recent failures only	`{job="aoi-inference", inspection_result="FAIL"} | json`
Latency anomaly	`{job="aoi-inference"} | json | inference_latency_ms > 500`
Low confidence events	`{job="aoi-inference"} | json | confidence_score < 0.56`
Board-level isolation	`{job="aoi-inference", pcb_id= "PCB-DEAD-.*"} | json`
Defect-type concentration	`{job="aoi-inference", defect_type="SOLDER_BRIDGE"}`

5.9 Measured Results Summary

The current repository already contains one Loki-backed report generated after executing the anomaly suite. Within a 60-minute lookback window, the system recorded 896 events: 451 `PASS` events and 445 `FAIL` events, for an overall fail rate of 49.7%. The defect distribution in that report was led by `NO_DEFECT` (466 events), followed by `SOLDER_BRIDGE` (180), `MISSING_COMPONENT` (95), `INSUFFICIENT_SOLDER` (70), `BENT_LEAD` (66), `MISALIGNMENT` (14), and `SOLDER_BALL` (5).

These aggregate values should not be read as factory-level quality rates, because the suite deliberately mixes normal and abnormal traffic. Their purpose is to demonstrate that the monitoring stack preserves a sufficiently rich event history to reconstruct both system-wide behavior and defect composition after the experiments finish.

The suite-level totals are complemented by the scenario-by-scenario quantitative profile in Table 6. That table is derived from the experiment generators themselves: event counts are deterministic from the batch construction logic, while fail rates and value ranges reflect the configured scenario distributions and randomized jitter used during execution.

Table 5: Suite-Level Results From `anomaly_results.md`

Metric	**Observed Value**
Total events logged	896
PASS events	451
FAIL events	445
Overall fail rate	49.7%
Most frequent defect label	`NO_DEFECT` (466)
Most frequent failure label	`SOLDER_BRIDGE` (180)
Provisioned alert rules	3
Observed alert states in capture	2 firing, 1 pending

5.10 Observed Outcome

The evidence collected so far supports the thesis claim. Each anomaly family produced a distinct visual and queryable signature, and the same structured log stream supported both dashboard-level summaries and targeted forensic queries. The baseline remained stable, fail-rate spikes became obvious in short windows, latency anomalies remained visible even when inference outputs continued to arrive, and recovery scenarios produced clear state transitions over time.

Table 6: Per-Scenario Quantitative Profile From Experiment Generators

ID	Events	Fail Profile	Latency Profile	Confidence Profile
S01	120	30% FAIL baseline	20–45 ms normal	0.76–1.00 high confidence
S02	100	90% FAIL spike	22–36 ms stable	0.77–0.93 high confidence
S03	70	40% FAIL mixed	escalating to ∼2000 ms	0.78–0.97 stable confidence
S04	72	50% FAIL mixed	25–45 ms normal	0.40–0.55 low confidence
S05	88	∼85% FAIL	22–40 ms stable	0.78–0.94 with one dominant defect label
S06	120	3 dead boards, 24 guaranteed FAIL events	20–50 ms mixed	0.75–0.99 mixed confidence
S07	90	phase shift: 90% FAIL to near-baseline	fault phase: 800–1800 ms; recovery phase: 20–42 ms	0.75–0.98 across fault and recovery phases
S08	120	3 full fault bursts inside mostly normal traffic	20–50 ms stable	0.79–0.98 mixed confidence
S09	140	gradual rise from 20% to 75% FAIL	20–45 ms stable	0.95 down to 0.40 drift profile

Table 7: Scenario Outcome Summary

ID	Primary Signal	Alert / Query	Observed Outcome
S01	baseline traffic	dashboard baseline	healthy 70/30 traffic and sub-50 ms latency established
S02	fail-rate spike	fail-rate alert	`FAIL` dominates, alert threshold exceeded clearly
S03	latency spike	latency alert	latency climbs into 500–2500 ms anomaly range
S04	confidence drop	confidence query	confidence remains in the 0.40–0.55 uncertainty band
S05	defect-type flood	defect filter	one defect label dominates the failure breakdown
S06	board failure	board filter	`pcb_id` isolates dead boards in one query
S07	recovery	alert resolution	fault period followed by visible return to healthy behavior
S08	intermittent spikes	narrow time-window query	short bursts visible in logs though weaker in coarse aggregates
S09	gradual degradation	confidence trend	slow confidence decline and fail-rate growth remain visible

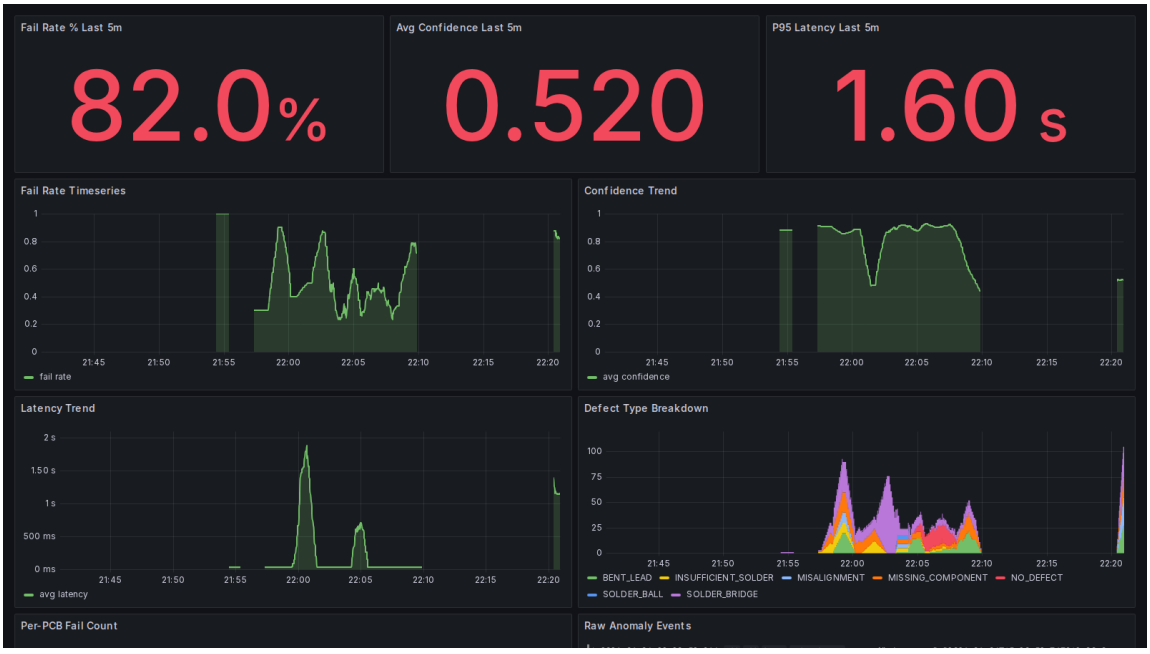

Figure 14: Overview of the anomaly-detection dashboard used in the experiment suite.

6 Evaluation and Discussion

6.1 Evaluation Criteria

The monitoring system should be evaluated using practical criteria rather than only implementation completeness. The key questions are:

- Can abnormal conditions be distinguished from baseline behavior?
- Are latency changes visible quickly enough to support action?
- Do the logs preserve enough detail for post-event investigation?
- Are dashboard queries simple enough to maintain locally?

6.2 Evaluation Logic

The evaluation in this report is based on whether the observability stack can turn raw inference events into meaningful operational evidence. In other words, the central issue is not whether the underlying detector is perfect, but whether the monitoring system makes important changes in service behavior visible and interpretable.

The anomaly suite provides a structured basis for that evaluation:

- S01 defines the baseline for normal operation.
- S02 and S03 test threshold-type anomalies, where a metric suddenly crosses a clear operational boundary.
- S04 and S09 test softer model-health signals based on confidence.
- S05 and S06 test whether the log schema is rich enough to support targeted investigation.
- S07 and S08 test temporal behavior, including recovery and intermittency.

6.3 Strengths of the Proposed Approach

The main strength of the approach is its pragmatism. The monitoring pipeline is small, understandable, and aligned with the repository that already exists. It does not depend on large cloud infrastructure or hidden instrumentation layers. Structured logs provide a stable bridge between AI inference and operational analytics, while the AOI workstation gives the same event data a human review context.

6.4 Findings From the Executed Scenario Suite

The current report and screenshots already support four concrete findings.

First, threshold-based anomalies should be detected quickly. A defect rate above 60–80% or an average latency above 500 ms should become visible within a short scrape interval and should trigger corresponding dashboard attention. In the collected Grafana capture, the dashboard displayed 82.0% fail rate, 0.520 average confidence, and 1.60 s P95 latency during an anomaly window, while the alerting page showed two rules firing and one pending.

Second, structured fields should make targeted diagnosis easier. The presence of `pcb_id`, `defect_type`, `confidence_score`, and `inference_latency_ms` allows the same event stream to support both summary dashboards and detailed investigation.

Third, the monitoring stack should be able to distinguish between different classes of abnormal behavior. A fail-rate spike, a latency spike, a low-confidence storm, and a board-specific failure do not have the same operational meaning, and the system should make those differences visible.

Fourth, time-series behavior matters. Recovery after anomaly and gradual degradation are important because industrial systems do not fail only in sudden, permanent ways. A useful monitoring system must also capture return to normality and slow deterioration over time.

6.5 Reporting Structure for Final Results

The final submitted version of this report should present scenario evidence in a consistent structure so that the reader can compare anomaly types without switching interpretive frameworks between sections. For each selected scenario, the results discussion should include the visual dashboard evidence, the query used to isolate the behavior, the corresponding numeric summary, and a short interpretation of why that behavior matters operationally in AOI inspection.

In practice, that means each scenario write-up should still include:

- one Grafana screenshot or chart excerpt;
- one supporting LogQL query;
- a short numeric summary, such as fail rate, average latency, or confidence range; and
- one sentence interpreting the operational meaning of the result.

This format is appropriate for a bachelor internship report because it is both technical and readable. It shows not only what the dashboard displayed, but also why that display matters in an AOI context.

6.6 Limitations

Several limitations remain. The current design is log-centric and does not yet include tracing across multiple services. Alert policies, long-term retention, and model-drift monitoring are also outside the present scope. In addition, the evaluation scenarios are controlled rather than derived from a deployed factory line. These limitations should be stated clearly in the final report to avoid overclaiming maturity.

The most important limitation is external validity. The anomaly suite was designed to test whether the monitoring stack reacts correctly to known conditions, not to estimate how frequently these conditions occur in a real factory. Scenario generators intentionally exaggerate some signals, such as a 90% fail spike or an abrupt confidence collapse, so that dashboard and alert behavior can be inspected clearly. This is appropriate for a system-design and monitoring thesis, but it means the resulting figures should be interpreted as validation of observability mechanics rather than direct evidence of production rates in the host company.

A second limitation is that the current system uses a local AOI workstation and synthetic event injection rather than a full online production line with hardware-triggered image acquisition, operator shift variation, and delayed or partial human labels. In a deployed industrial setting, additional factors would need to be monitored, including equipment state, label latency, board-mix changes, and line-level throughput constraints. The present report therefore demonstrates a strong monitoring prototype and experiment harness, but not a completed factory observability rollout.

6.7 Future Work

Future work can extend the platform in four directions:

- integrate a production-grade detector in place of mock inference;
- add model version, runtime, and hardware metadata to each event;
- define alert thresholds for failure spikes and latency excursions; and
- compare log-based monitoring with richer metrics or tracing systems.

6.8 Chapter Conclusion

The evaluation perspective of this report is that useful AI monitoring does not require a giant platform at the first stage. What it requires is a disciplined event schema, a reproducible log pipeline, and dashboards that answer concrete operational questions. For the AOI case study, this is enough to build a strong foundation for anomaly detection, system debugging, and future model-service research.

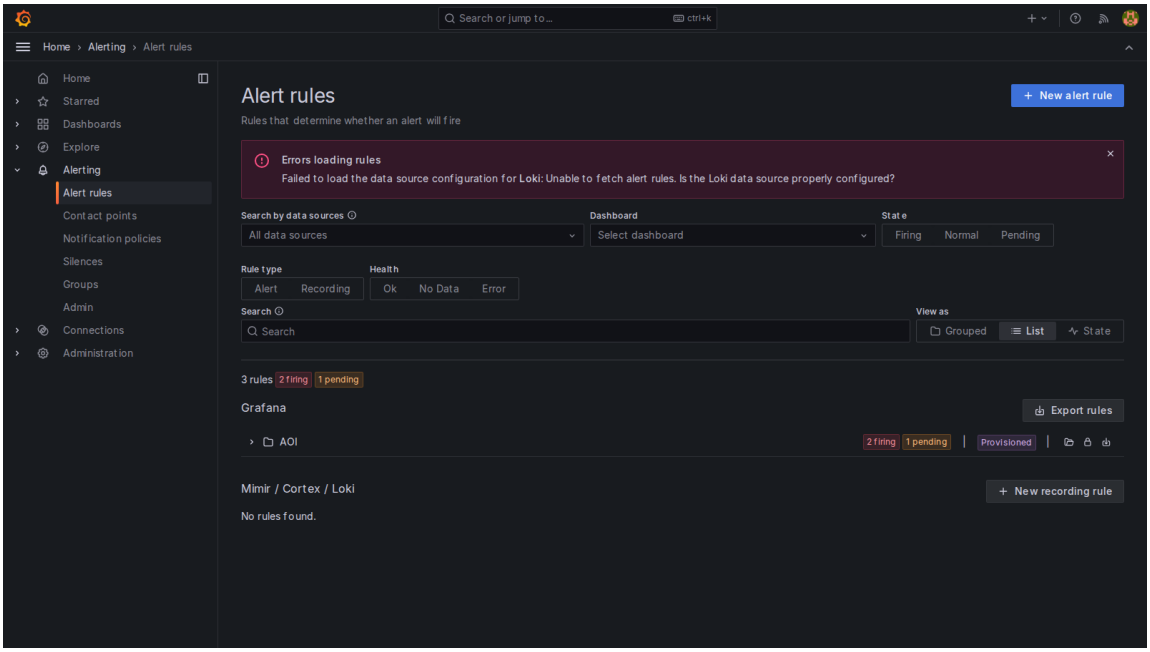

Figure 15: Alerting evidence confirms that the monitoring design supports actionable fault detection.

Conclusion

This bachelor internship report reframes the AOI project as an operational AI system whose inference behavior must be observable, measurable, and reviewable. By combining structured event logging with a lightweight Loki–Promtail–Grafana stack, the system supports a practical path for detecting failure concentration, latency degradation, and event-flow anomalies. The implemented experiment harness, version-controlled alert rules, and Loki-backed report generation show that the monitoring workflow is not only conceptual but executable. Although its scale is intentionally small compared with platforms such as Netflix Mantis or specialized inference servers such as NVIDIA Triton, the design answers the actual research question of this internship: how to make an AOI inference service operationally visible, diagnosable, and testable in practice.

In terms of the report's four guiding questions, the work built a structured logging and monitoring system for AOI inference events; it was needed because model accuracy alone does not provide the operational visibility required for industrial inspection; it was implemented through a FastAPI backend, JSONL log emission, Promtail ingestion, Loki query storage, Grafana dashboards, and a reproducible anomaly-test harness; and it achieved the main intended result of making normal behavior, fail-rate spikes, latency anomalies, confidence drops, recovery, and gradual degradation visible through the same observability stack.

References

[1] B. Beyer, C. Jones, J. Petoff, and N. Murphy, *Site Reliability Engineering*. O'Reilly Media, 2016.

[2] R. Turnbull, *Grafana 2nd Edition*. Packt, 2018.

[3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden Technical Debt in Machine Learning Systems," in *Advances in Neural Information Processing Systems 28*, 2015.

[4] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction," in *Proc. IEEE International Conference on Big Data*, 2017.

[5] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software Engineering for Machine Learning: A Case Study," in *Proc. IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, 2019.

[6] S. Shankar and A. G. Parameswaran, "Towards Observability for Production Machine Learning Pipelines," *PVLDB*, vol. 15, no. 13, pp. 4015–4022, 2022.

[7] S. Shankar, R. Garcia, J. M. Hellerstein, and A. G. Parameswaran, "Operationalizing Machine Learning: An Interview Study," in *Proc. IEEE/ACM 1st Workshop on AI Engineering – Software Engineering for AI*, 2021.

[8] Grafana Labs, "Loki: like Prometheus, but for logs," online documentation, `https://grafana.com/oss/loki`.

[9] S. Ramírez, "FastAPI," online documentation, https://fastapi.tiangolo.com/.

[10] M. Treveil and A. Omont, *Introducing MLOps*. O'Reilly Media, 2020.

[11] Netflix, "Mantis," official documentation, https://netflix.github.io/mantis/.

[12] Netflix, "Mantis Use Cases," official documentation, https://netflix.github.io/mantis/getting-started/use-cases/.

[13] Netflix, "Netflix Open Source Software Center," https://netflix.github.io/.

[14] NVIDIA, "Triton Inference Server Metrics," official documentation, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/metrics.html.